

GLOBALRETAIL Direction Supply Chain

Stock Predict

Plateforme de prédiction des stocks basée sur le Machine Learning

DOSSIER DE CONCEPTION

Projet 2 Conception fonctionnelle, technique, UX/UI et exigences non fonctionnelles

Équipe projet (SCRUM)

Paul (Scrum Master / dev) · Sarah (Product Owner / dev) · Hugo (dev data) · Benjamin (dev back) ·
Steven (dev front)

Juillet 2026 Version 1.0 Proof of Concept (POC)

Sommaire

Sommaire	2
1. Présentation du sujet et de la démarche	3
1.1 Le commanditaire et son besoin	3
1.2 Périmètre retenu : un Proof of Concept	3
1.3 Démarche de conception	3
2. Conception fonctionnelle	5
2.1 Acteurs et rôles	5
2.2 Diagramme de cas d'utilisation (UML)	5
2.3 Cas d'utilisation détaillés	6
2.4 Parcours utilisateurs	7
2.5 Règles de gestion	8
3. Conception technique	9
3.1 MCD Modèle conceptuel de données	9
3.2 MLD Modèle logique de données	10
3.3 MPD Modèle physique de données (PostgreSQL)	10
4. Conception UX / UI	12
4.1 Principes d'interface	12
4.2 Maquettes fonctionnelles	12
4.3 Accessibilité (RGAA)	15
5. Exigences non fonctionnelles	16
5.1 Sécurité OWASP Top 10	16
5.2 Performance	16
5.3 Scalabilité (documentée)	16
5.4 Maintenabilité	17
Annexe Livrables joints	18

1. Présentation du sujet et de la démarche

1.1 Le commanditaire et son besoin

Le groupe international **GLOBALRETAIL** est un acteur majeur de la grande distribution, présent dans plus de 25 pays et exploitant plusieurs dizaines d'entrepôts logistiques qui approvisionnent magasins physiques et plateformes de e-commerce. Son activité est marquée par une forte volatilité de la demande, des délais fournisseurs variables, des enjeux géopolitiques pesant sur les chaînes d'approvisionnement et une pression croissante sur les coûts logistiques. Cette situation justifie la mise en place d'un logiciel de gestion et de prédiction de stock.

La direction Supply Chain souhaite une plateforme capable de **prédire les niveaux de stock** afin de potentiellement limiter trois phénomènes coûteux :

- les ruptures de stock (ventes perdues, clients insatisfaits)
- le surstock (immobilisation financière, coûts de stockage)
- les pertes financières qui en résultent.

La solution s'appuie sur la collecte de données massives issues des systèmes logistiques existants, sur des modèles de Machine Learning (régression, séries temporelles) et sur une interface de pilotage destinée aux responsables logistiques.

1.2 Périmètre retenu : un Proof of Concept

Le présent projet est un **POC (Proof of Concept)** : il démontre la faisabilité de la chaîne complète ingestion de données, prédiction, restitution sur un périmètre volontairement réduit, tout en documentant la trajectoire vers l'échelle cible. Les choix de cadrage suivants ont été actés avec le commanditaire :

- **Deux pays pilotes** : France et Allemagne. L'interface permet de basculer de l'un à l'autre ; la généralisation aux 25 pays est traitée au niveau de l'architecture (données rattachées à un référentiel pays) mais non déployée.
- **Mondialisation via API externes** : la gestion des TVA et des règles douanières par zone géographique sera assurée par des API tierces spécialisées ; le POC se limite à stocker le taux de TVA par pays et à prévoir les points d'intégration.
- **PostgreSQL** comme SGBD : le volume de données (plusieurs millions de lignes de ventes par nuit) exclut SQLite ; PostgreSQL apporte partitionnement natif, index avancés et réplication. L'archivage froid à grande échelle (format Apache Iceberg sur stockage objet) est documenté mais hors périmètre POC.
- **Conteneurisation Docker** sans orchestration : Kubernetes est mentionné comme cible d'industrialisation pour souligner la dimension du projet, mais reste « sur le papier » au stade POC.
- **Pas de table d'historisation** : la conservation des états passés est assurée par une stratégie de sauvegarde en quasi temps réel (archivage WAL + PITR, règle 3-2-1) plutôt que par des tables d'historique applicatives.

Enfin, l'ingestion automatisée est réalisée par des **scripts shell planifiés via CRON**, le nettoyage écarte les colonnes non mappées au schéma, et la visualisation s'appuie sur la librairie **Chart.js**.

1.3 Démarche de conception

La conception a été menée en équipe SCRUM de cinq développeurs (Paul, Sarah, Hugo, Benjamin, Steven) sur un planning global de six mois. Elle suit une progression classique : du **fonctionnel** (qui fait quoi ? acteurs, cas d'utilisation, parcours, règles de gestion) vers le **technique** (comment ? modèle de données Merise MCD → MLD → MPD, cas d'utilisation UML), puis vers l'**UX/UI** (maquettes fonctionnelles, accessibilité RGAA) et enfin les **exigences non fonctionnelles** (sécurité OWASP, performance, scalabilité, maintenabilité). Chaque section du présent dossier correspond à un livrable attendu du Projet 2 « Conception ».

2. Conception fonctionnelle

2.1 Acteurs et rôles

Acteur	Rôle dans la plateforme	Profil applicatif
Responsable logistique	Pilote les stocks : consulte KPI, stocks, prévisions, traite les alertes de rupture, exporte des données.	RESPONSABLE
Data analyst (supply chain)	Contrôle la qualité des données synchronisées, analyse les prévisions et exporte les données pour les métiers.	ANALYSTE
Administrateur	Gère les comptes et les droits, consulte les journaux d'audit, supervise les traitements.	ADMIN
Ordonnanceur (CRON)	Acteur système : déclenche chaque nuit la synchronisation des données via les API sources, puis le recalcul des prévisions ML et la génération des alertes.	(système)

2.2 Diagramme de cas d'utilisation (UML)

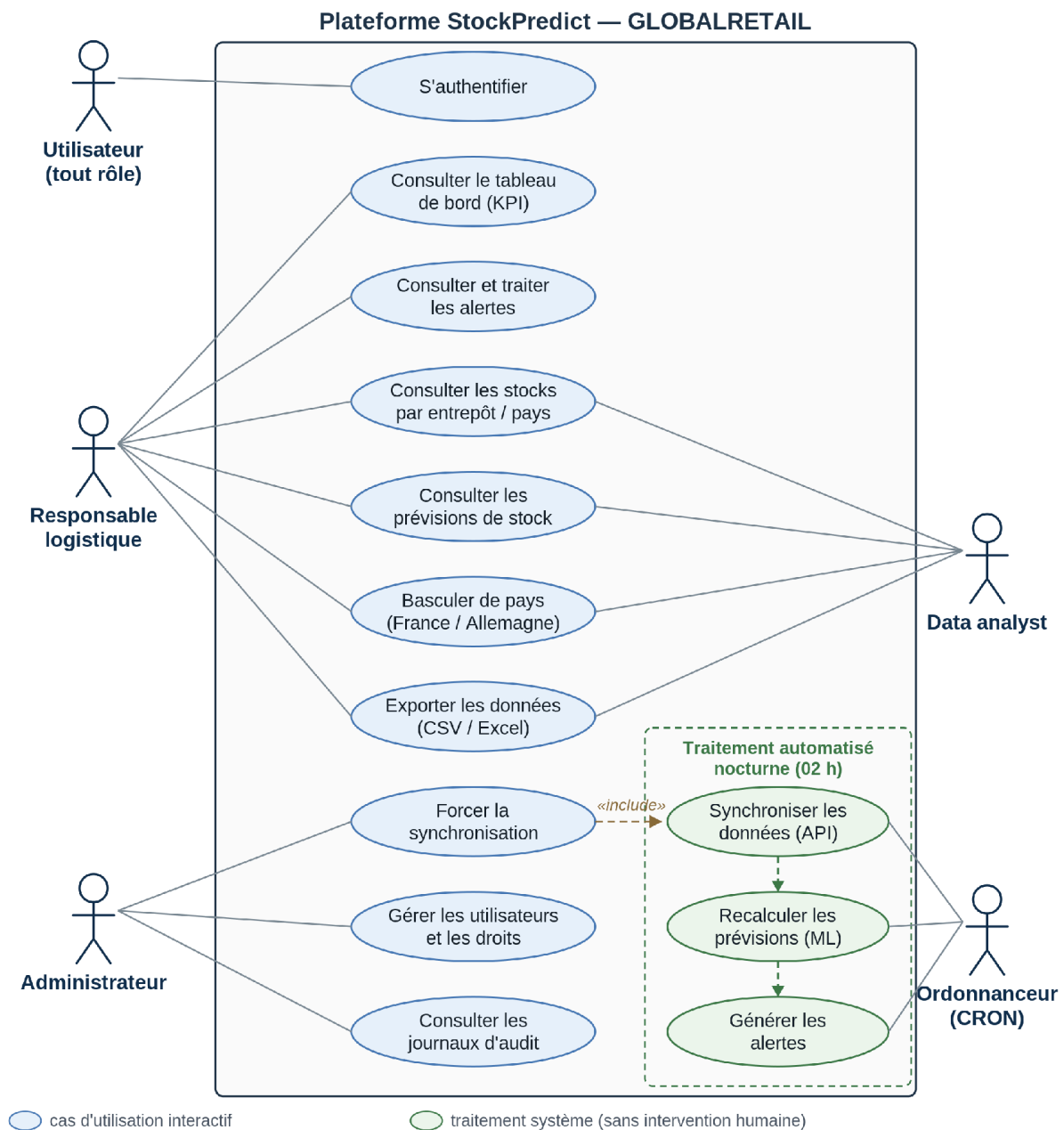


Figure 1 Diagramme UML de cas d'utilisation de la plateforme StockPredict

Toutes les fonctions consultatives passent par l'authentification préalable («include» implicite depuis chaque cas d'utilisation vers « S'authentifier », allégé sur le schéma pour la lisibilité). De cette authentification découle ensuite le rôle de l'utilisateur connecté et donc ses droits d'actions de visibilité. L'ordonnanceur CRON est modélisé comme acteur secondaire : il enchaîne chaque nuit la synchronisation des données via les API des systèmes sources — seule voie d'alimentation de la plateforme —, puis le recalcul des prévisions et la génération des alertes. L'administrateur peut uniquement forcer l'exécution de cette chaîne, jamais saisir ni déposer de données.

2.3 Cas d'utilisation détaillés

UC-01 S'authentifier

Rubrique	Description
Acteur(s)	Tous les acteurs humains
Préconditions	L'utilisateur dispose d'un compte actif créé par l'administrateur.
Scénario nominal	1. L'utilisateur saisit e-mail et mot de passe. 2. Le système vérifie la correspondance avec le hash stocké (bcrypt). 3. Une session est ouverte (JWT token, expiration 8 h). 4. La connexion est journalisée (date, IP). 5. L'utilisateur est redirigé vers le tableau de bord.
Scénarios alternatifs / erreurs	3a. Identifiants invalides : message générique « identifiants incorrects », compteur d'échecs incrémenté. 3b. Cinq échecs consécutifs : compte verrouillé 15 min et événement journalisé.
Postconditions	L'utilisateur est authentifié ; ses droits dépendent de son rôle.
Règles de gestion liées	RG-12, RG-13, RG-14, RG-15

UC-02 Consulter les prévisions de stock

Rubrique	Description
Acteur(s)	Responsable logistique, Data analyst
Préconditions	Utilisateur authentifié ; au moins un calcul de prévision disponible.
Scénario nominal	1. L'utilisateur sélectionne un produit, un entrepôt et un horizon (7, 30, 60 ou 90 jours). 2. Le système affiche la courbe historique, la prévision, l'intervalle de confiance à 95 % et le seuil d'alerte. 3. Une synthèse tabulaire indique les dates de rupture probables et une recommandation de commande.
Scénarios alternatifs / erreurs	2a. Aucune prévision pour ce couple produit/entrepôt : le système propose de lancer un calcul. 2b. Données historiques insuffisantes (< 60 jours) : avertissement de fiabilité réduite.
Postconditions	Aucune modification de données ; consultation journalisée.
Règles de gestion liées	RG-06, RG-07, RG-08

UC-03 Forcer l'import des données (API)

Rubrique	Description
Acteur(s)	Administrateur (forçage manuel) ; Ordonnanceur CRON (exécution planifiée nocturne)
Préconditions	Utilisateur authentifié avec le rôle ADMIN ; sources API configurées et joignables.
Scénario nominal	1. L'utilisateur appuie sur le bouton de refresh forcé 2. Le système requête les API 3. Les données sont vérifiées et validées 4. une notification de succès apparaît
Scénarios alternatifs / erreurs	2a. API injoignable, authentification refusée ou délai dépassé : synchronisation abandonnée après 3 tentatives, statut « Échec — API » journalisé et notification à l'administrateur. 3a. Données non conformes au schéma : les lignes fautives sont rejetées, comptabilisées et exportables (RG-02). 4a. Erreur d'insertion : transaction annulée, aucune donnée partiellement intégrée (RG-03).
Postconditions	Les données sont disponibles pour le calcul des prévisions ; l'import figure dans l'historique.
Règles de gestion liées	RG-01, RG-02, RG-03, RG-14

UC-04 Consulter et traiter les alertes de rupture

Rubrique	Description
Acteur(s)	Responsable logistique
Préconditions	Utilisateur authentifié ; alertes générées par le dernier calcul de prévision.
Scénario nominal	1. L'utilisateur ouvre la liste des alertes ouvertes (sous seuil, rupture prévue, surstock). 2. Il consulte le détail : produit, entrepôt, date de rupture estimée, recommandation. 3. Il marque l'alerte « traitée » (commande passée) ou « ignorée » (avec motif). 4. Le changement de statut est journalisé.
Scénarios alternatifs / erreurs	3a. L'alerte concerne un produit dont une commande fournisseur est déjà en cours : le système le signale et propose de lier l'alerte à la commande.
Postconditions	L'alerte change de statut ; les KPI du tableau de bord sont mis à jour.
Règles de gestion liées	RG-09, RG-10, RG-14

UC-05 Basculer de pays (France / Allemagne)

Rubrique	Description
Acteur(s)	Responsable logistique, Data analyst
Préconditions	Utilisateur authentifié.
Scénario nominal	1. L'utilisateur sélectionne un pays dans le sélecteur d'en-tête. 2. L'ensemble des vues (KPI, stocks, prévisions, alertes) est refiltré sur les entrepôts du pays choisi. 3. Les montants affichent la TVA du pays (référentiel PAYS).
Scénarios alternatifs / erreurs	2a. Vue « groupe » : un mode « Tous pays » agrège les indicateurs des deux pays pilotes.
Postconditions	Le contexte pays persiste pendant toute la session.
Règles de gestion liées	RG-04, RG-05

2.4 Parcours utilisateurs

Parcours 1 La revue matinale du responsable logistique

8 h 30 : Paul, responsable logistique France, se connecte. Le **tableau de bord** affiche immédiatement les quatre KPI du jour : 128 produits sous seuil, 34 ruptures prévues sous 7 jours, 96,4 % de disponibilité, 1,2 M€ de surstock. Le bouton de notification avec compteur lui signale trois cas critiques. Il ouvre l'alerte du produit REF-4512 (Lait UHT 1L, entrepôt Lyon-Sud) : la prévision annonce une rupture dans 4 jours, la recommandation propose une commande de 400 unités avant J+10. Paul vérifie la courbe de prévision (intervalle de confiance, saisonnalité), constate qu'aucune commande fournisseur n'est en cours, déclenche la commande dans son ERP, puis marque l'alerte « traitée ». Avant sa réunion, il bascule sur l'Allemagne pour vérifier que Berlin-Ouest ne présente pas la même tension, et exporte la liste des produits critiques en CSV pour son homologue allemand. Durée totale : moins de dix minutes ; chaque action est journalisée.

Parcours 2 L'intégration de données du data analyst

9 h : Sarah, data analyst, ouvre l'écran Synchronisation pour contrôler le traitement de la nuit. Les deux premières sources sont au vert — 4,18 M de lignes de ventes et 128 470 lignes de stocks intégrées — mais la source « Portail Fournisseurs » est en échec depuis deux exécutions : l'API n'a pas répondu dans le délai imparti. Elle en mesure l'impact : les délais d'approvisionnement affichés datent du 28/07, ce qui dégrade la fiabilité des prévisions sur les produits concernés. Elle signale l'incident à Steven, administrateur, qui relance une synchronisation forcée depuis le même écran ; l'exécution manuelle est journalisée à son nom (RG-14). Sarah télécharge ensuite le fichier des 1 843 lignes rejetées par l'ERP Ventes, identifie deux références produit absentes du référentiel, et ouvre une demande de correction auprès de l'équipe ERP. Aucune donnée n'est saisie ni déposée manuellement dans la plateforme : le rôle de l'analyste est de surveiller, qualifier et corriger à la source.

2.5 Règles de gestion

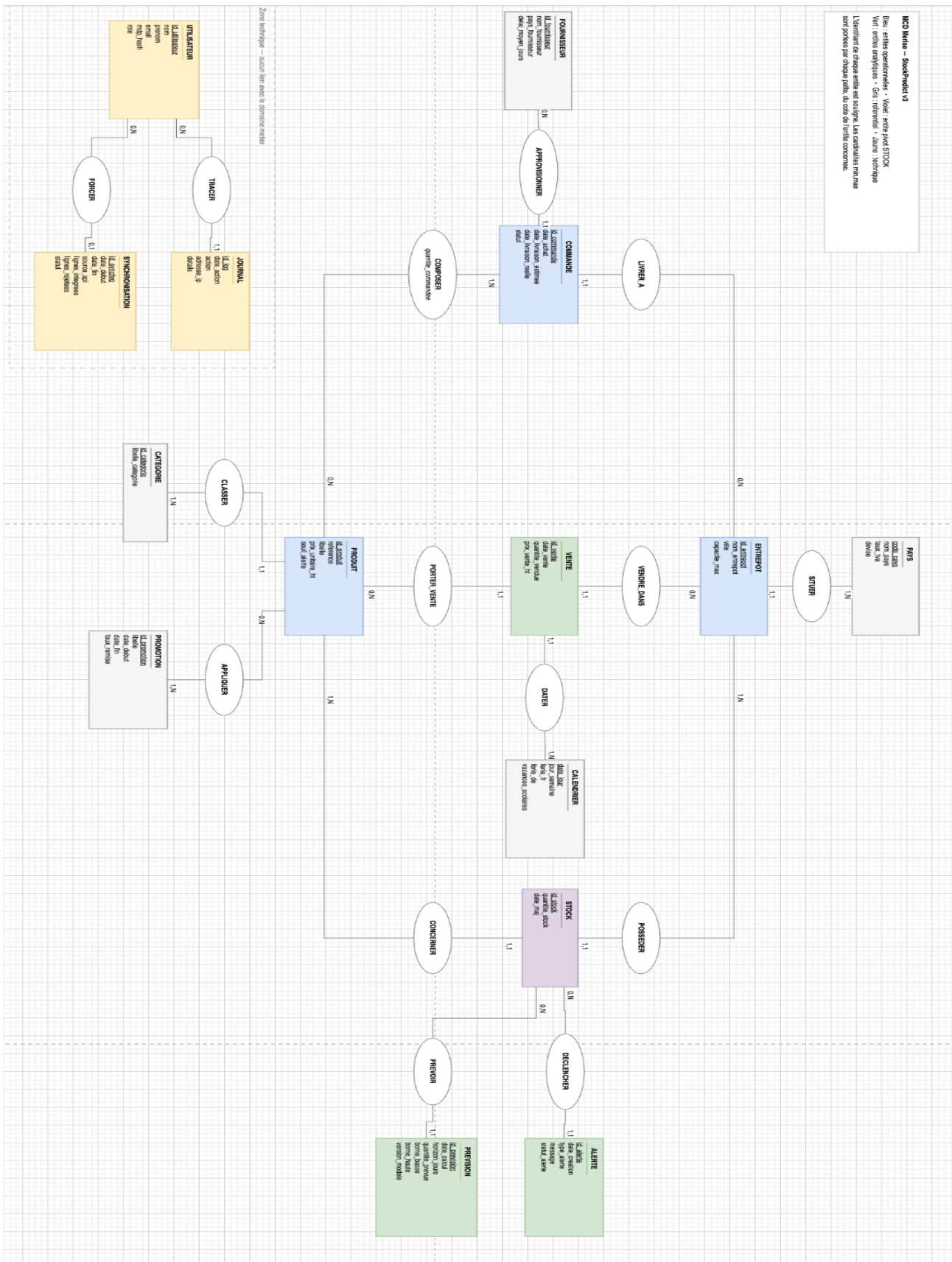
ID	Règle de gestion
RG-01	Tout import ne conserve que les colonnes mappées au schéma cible ; les autres sont ignorées et comptabilisées dans le compte rendu.
RG-02	Une ligne importée est rejetée si : type invalide, quantité négative, référence produit ou entrepôt inconnue, doublon strict.
RG-03	Un import est atomique : en cas d'erreur bloquante, aucune ligne n'est conservée (rollback).(ACID)
RG-04	Chaque entrepôt est rattaché à un et un seul pays ; tout indicateur est agrégeable par entrepôt, par pays ou par groupe de pays.
RG-05	Le taux de TVA appliqué à la valorisation est celui du référentiel PAYS ; les règles douanières relèvent d'API externes (hors POC).
RG-06	Une prévision porte sur un couple produit × entrepôt et un horizon parmi 7, 30, 60 ou 90 jours ; elle comporte toujours un intervalle de confiance à 95 %.
RG-07	Les prévisions intègrent les facteurs saisonniers (jour de semaine, mois, périodes promotionnelles et fêtes calendaires).

ID	Règle de gestion
RG-08	Toute prévision est traçable : date de calcul et version du modèle sont conservées.
RG-09	Une alerte SOUS_SEUIL est créée quand le stock passe sous le seuil d'alerte du produit ; une alerte RUPTURE_PREVUE quand la borne basse de prévision atteint zéro dans l'horizon de 7 jours.
RG-10	Une alerte ne peut être fermée que par un changement de statut explicite (« traitée » ou « ignorée » avec motif), par un utilisateur habilité.
RG-11	Le seuil d'alerte d'un produit est un entier positif ou nul, modifiable uniquement par un profil RESPONSABLE ou ADMIN.
RG-12	Les mots de passe sont stockés exclusivement sous forme de hash renforcé (bcrypt/argon2) ; aucune transmission ni journalisation en clair.
RG-13	Les droits sont portés par trois rôles (RESPONSABLE, ANALYSTE, ADMIN) ; toute action est refusée par défaut si le rôle ne l'autorise pas explicitement.
RG-14	Toute action significative (connexion, import, export, alerte émise ou traitée, administration) est journalisée : horodatage, utilisateur ou système, IP, détails.
RG-15	Cinq échecs de connexion consécutifs verrouillent temporairement le compte pendant 15 minutes ; l'événement est journalisé.

3. Conception technique

La modélisation des données suit la méthode **Merise** : modèle conceptuel (MCD), puis logique relationnel (MLD), puis physique (MPD) ciblant PostgreSQL. Le diagramme UML de cas d'utilisation figure en section 2.2.

3.1 MCD Modèle conceptuel de données



lien du draw.io :

https://drive.google.com/drive/folders/1OCzpXSpBX0xQgQdh_BmrPENEGMktitDv

Figure 2 MCD Merise : entités, associations et cardinalités

Le cœur du modèle est le couple PRODUIT × ENTREPOT : l'association porteuse STOCKER matérialise le stock courant (quantité, date de mise à jour), tandis que VENTE, PREVISION et ALERTE se rattachent chacune à un produit et à un entrepôt. L'approvisionnement est couvert par FOURNISSEUR, COMMANDE (dates d'achat et de livraison estimée puis réelle, exigées par le cahier des charges) et l'association porteuse COMPOSER (quantités commandées). Le référentiel PAYS (taux de TVA, devise) porte le pilotage multi-pays. Deux entités soutiennent directement la règle RG-07 : CALENDRIER, dimension temps qui qualifie chaque journée (jour de semaine, jours fériés distincts pour la France et l'Allemagne, vacances scolaires), et PROMOTION, qui décrit les périodes promotionnelles appliquées aux produits ; ces deux entités fournissent au modèle de prévision les variables explicatives calendaires et commerciales, connues également pour les dates futures. Enfin, SYNCHRONISATION trace chaque exécution d'alimentation (source interrogée, volumétrie intégrée et rejetée, statut, déclencheur automatique ou forçage par un administrateur), UTILISATEUR et JOURNAL portant la sécurité et la traçabilité exigées.

3.2 MLD Modèle logique de données

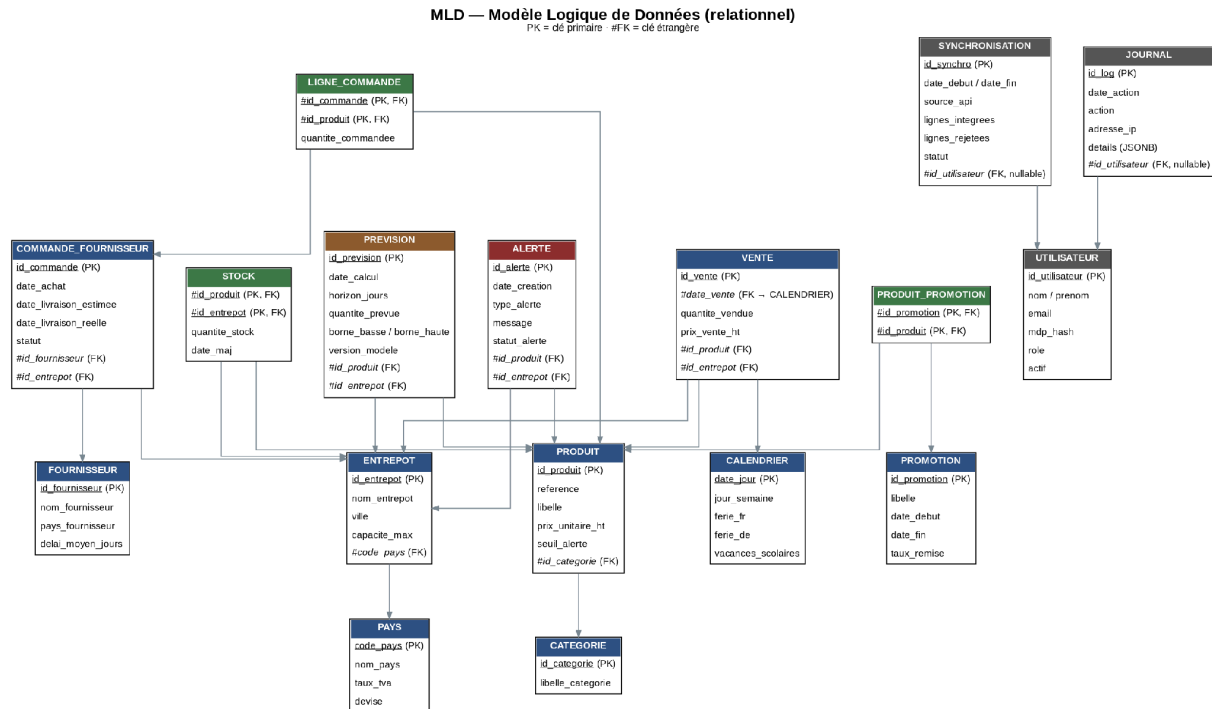


Figure 3 MLD relationnel : clés primaires (PK), clés étrangères (FK)

En notation relationnelle textuelle (clés primaires soulignées par convention, # = clé étrangère) :

PAYS (code_pays, nom_pays, taux_tva, devise)
 PRODUIT_PROMOTION (#id_promotion, #id_produit)
 PROMOTION (id_promotion, libelle, date_debut, date_fin, taux_remise)
 CALENDRIER (date_jour, jour_semaine, numero_semaine, mois, ferie_fr, ferie_de, vacances_scolaires)
 ENTREPOT (id_entrepot, nom_entrepot, ville, capacite_max, #code_pays)
 CATEGORIE (id_categorie, libelle_categorie)
 PRODUIT (id_produit, reference, libelle, prix_unitaire_ht, seuil_alerte, #id_categorie)
 STOCK (#id_produit, #id_entrepot, quantite_stock, date_maj)
 VENTE (id_vente, #date_vente, quantite_vendue, prix_vente_ht, #id_produit, #id_entrepot)
 FOURNISSEUR (id_fournisseur, nom_fournisseur, pays_fournisseur, delai_moyen_jours)
 COMMANDE_FOURNISSEUR (id_commande, date_achat, date_livraison_estimee, date_livraison_reelle, statut, #id_fournisseur, #id_entrepot)
 LIGNE_COMMANDE (#id_commande, #id_produit, quantite_commandee)
 PREVISION (id_prevision, date_calcul, horizon_jours, quantite_prevue, borne_basse, borne_haute, version_modele, #id_produit, #id_entrepot)
 ALERTE (id_alerte, date_creation, type_alerte, message, statut_alerte, #id_produit, #id_entrepot)
 UTILISATEUR (id_utilisateur, nom, prenom, email, mdp_hash, role, actif)
 SYNCHRONISATION (id_synchro, date_debut, date_fin, source_api, declencheur, lignes_integrees, lignes_rejetees, statut, #id_utilisateur)
 JOURNAL (id_log, date_action, action, adresse_ip, details, #id_utilisateur)

3.3 MPD Modèle physique de données (PostgreSQL)

Le MPD complet est livré sous forme de script SQL exécutable (**mpd_stockpredict.sql**, joint au dossier). Les choix physiques notables sont les suivants :

- **Traitement de gros volumes** : Capacité native à gérer plusieurs millions de lignes de ventes quotidiennement avec un historique cumulé important, sans dégradation des temps de réponse.
- **Partitionnement déclaratif** : Utilisation du partitionnement par plage (range partitioning) sur la colonne `date_vente` pour optimiser les insertions nocturnes et automatiser le cycle de vie des données (purge/archivage).
- **Indexation spécialisée** : Mise en œuvre d'index composites, partiels pour les alertes actives, et GIN pour les colonnes JSONB (journaux), assurant des temps de réponse ciblés.
- **Fiabilité ACID** : Respect strict des propriétés ACID, garantissant l'atomicité des imports de données (rollback automatique en cas d'erreur bloquante) et la cohérence transactionnelle indispensable au pilotage Supply Chain.
- **Robustesse opérationnelle** : Support natif de la réplication et facilité d'intégration avec des stratégies de sauvegarde éprouvées (WAL archiving, PITR), garantissant la continuité de service exigée.

3.4 Stack technique front-end et back-end

L'entreprise utilise habituellement Python pour le backend, choisi pour sa richesse en bibliothèques de traitement et manipulation de données (**Pandas**, **Scikit-learn**) ainsi que pour son intégration robuste avec **PostgreSQL**. Une grande partie de l'équipe maîtrisant déjà cet écosystème, cette technologie répond idéalement aux besoins de calculs du projet.

Back-end Headless (API REST)

Afin d'éviter la complexité d'une architecture multi-backends, l'ensemble du back-end sera développé en Python sous forme d'**API REST Headless** en utilisant le framework **FastAPI** (ou *Django REST Framework*). Ce choix permet de tirer parti des capacités de calcul de Python tout en fournissant une API moderne, typée, auto-documentée (Swagger) et performante pour alimenter le front-end.

Front-end

L'indexation des pages par les moteurs de recherche (SEO) n'étant pas requise pour cette application fonctionnelle, le choix s'est porté sur une architecture Single Page Application (SPA) basée sur **Vue.js 3** (couplé à **Vite**). Ce choix garantit un build léger, un développement rapide et une maintenance simplifiée sans la surcouche SSR d'un framework comme Nuxt.

Le front-end s'appuiera sur :

- **Shadcn-vue** (composants UI) pour accélérer le développement et garantir l'uniformité visuelle.
- **TanStack Table** pour la gestion performante et filtrable des tableaux de données.
- **Chart.js** pour la restitution graphique des indicateurs clés.

Cette séparation stricte via une API REST permet de découper totalement le front-end du back-end, offrant une interface réactive adaptée à la manipulation de volumétries importantes tout en s'appuyant sur un backend robuste et connu des équipes.

Base de données

PostgreSQL a été retenu comme SGBD pour sa capacité à gérer des volumétries massives et critiques, surpassant les limitations d'un système relationnel plus simple et s'intégrant parfaitement avec l'écosystème Python.

3.5 Coût global

L'estimation du coût de conception du POC repose sur une **équipe complète de 5 personnes** : 3 développeurs (Hugo, Benjamin, Steven) travaillant à plein temps, ainsi que Paul (Scrum Master) et Sarah (Product Owner) intervenant à mi-temps sur une période de 3 mois (soit 66 jours ouvrés par mois pour l'ensemble de l'équipe).

Le TJM (Taux Journalier Moyen) retenu pour l'ensemble des profils est de 450 € HT pour l'équipe de développement et 350 € HT pour le scrum master et product owner.

Rôle	Durée (jours)	TJM moyen (€)	Total (€)
Équipe développement (3 pers.)	198 (66 j/pers)	450	89 100
Scrum Master & Product Owner	66 (33 j/pers)	350	23 100
TOTAL GÉNÉRAL	264	-	112 200

Le coût total actualisé pour la phase de conception est de **112 200 € HT**. Cette estimation intègre l'encadrement agile (Scrum Master et Product Owner) en complément des ressources de développement pour assurer la cohérence fonctionnelle et technique du POC.

4. Conception UX / UI

4.1 Principes d'interface

L'interface vise un utilisateur métier non technicien : navigation latérale unique et constante, indicateurs synthétiques en tête d'écran, code couleur homogène (vert = conforme, orange = sous seuil, rouge = critique), et sélecteur de pays permanent dans l'en-tête (France / Allemagne) conformément à l'exigence multi-pays. Les maquettes ci-dessous, réalisées en wireframes moyenne fidélité (équivalent Figma), couvrent les cinq écrans du périmètre POC. Les graphiques seront implémentés avec la librairie [Chart.js](#).



Chart.js

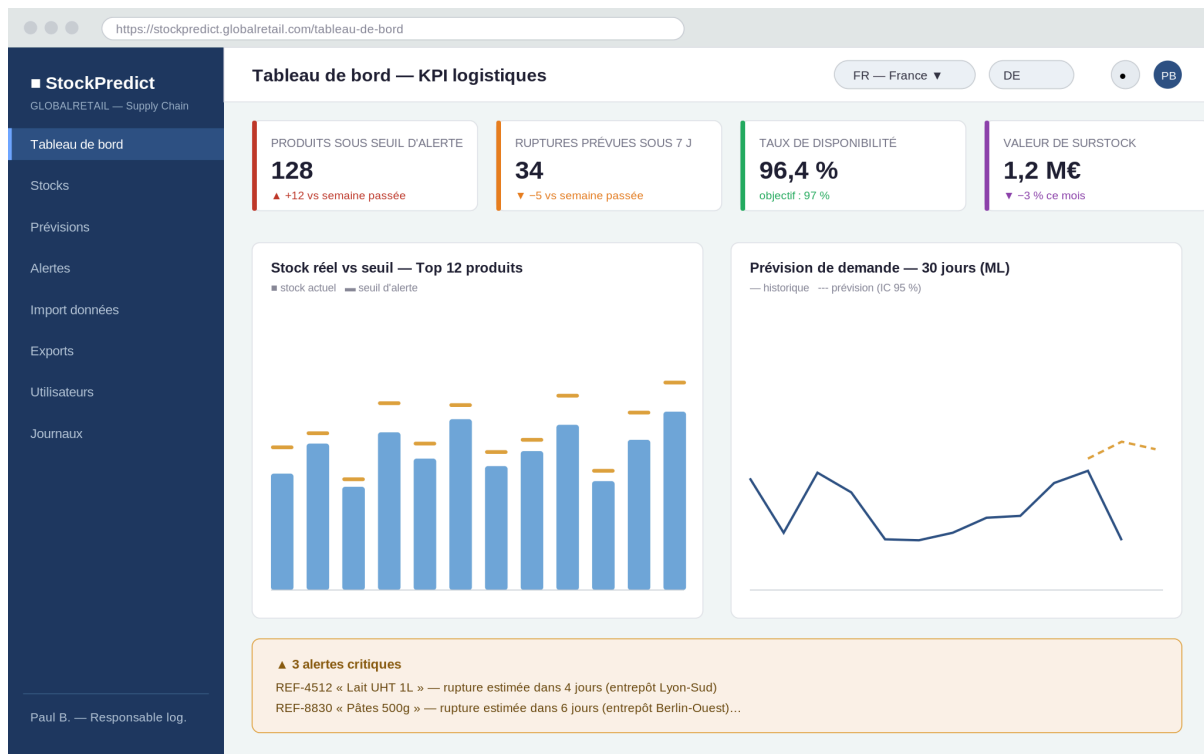


Figma

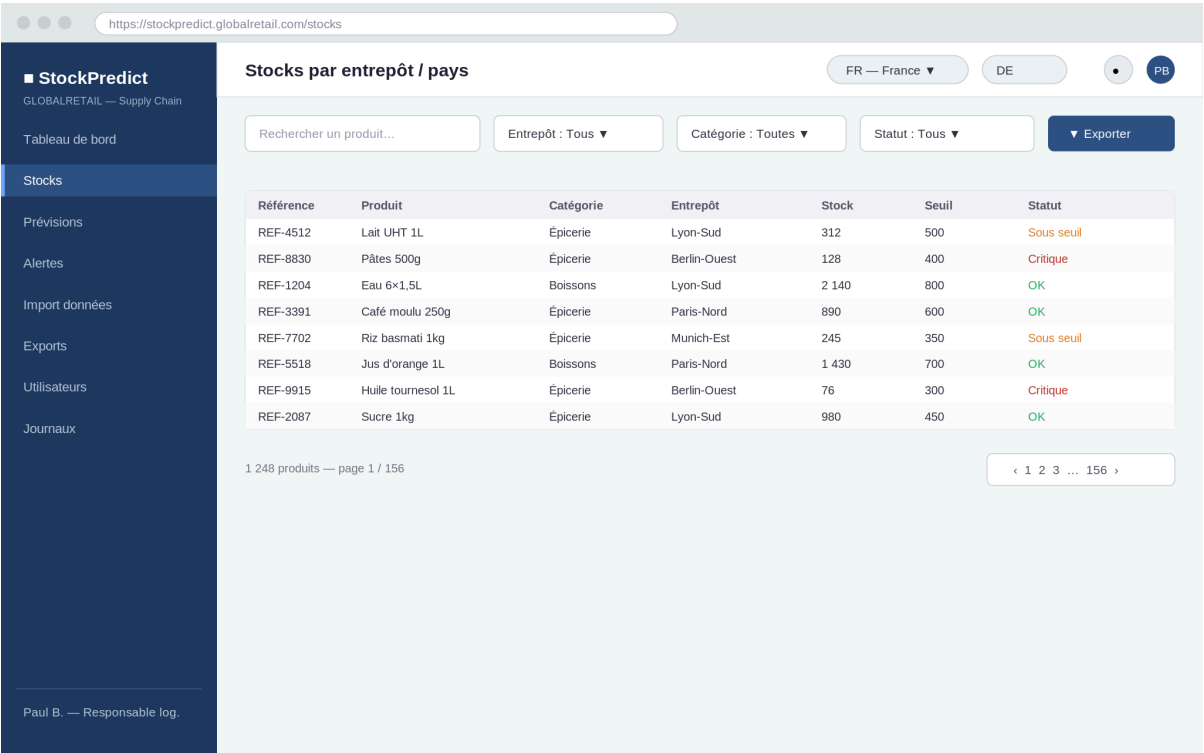
4.2 Maquettes fonctionnelles

A wireframe of a login page for 'StockPredict'. The page has a light blue background. At the top, there's a browser address bar showing 'https://stockpredict.globalretail.com/connexion'. The main content is a white card with rounded corners. Inside the card, the 'StockPredict' logo is at the top, followed by the tagline 'Plateforme de prédiction des stocks'. Below this are two input fields: 'Adresse e-mail' with the placeholder 'prenom.nom@globalretail.com' and 'Mot de passe' with a masked password '*****' and a 'voir' link. A dark blue 'Se connecter' button is below the password field. A link 'Mot de passe oublié ?' is at the bottom of the card. At the very bottom of the page, there's a small footer: 'Accès réservé — toutes les connexions sont journalisées'.

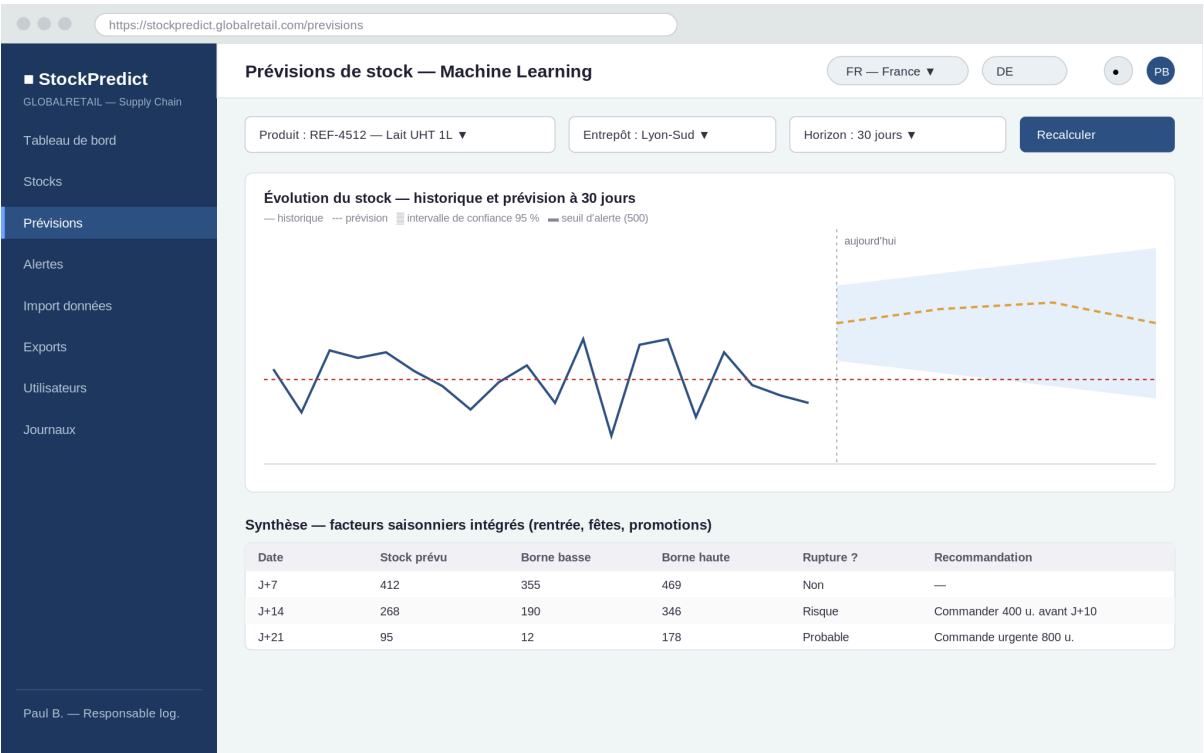
Maquette 1 Connexion : authentication, lien de récupération, mention de journalisation



Maquette 2 Tableau de bord : 4 KPI logistiques, stock vs seuil, prévision de demande, compteur de notifications et alertes critiques



Maquette 3 Stocks : recherche, filtres entrepôt/catégorie/statut, statut coloré par ligne, export



Maquette 4 Prévisions : courbe historique + prévision avec intervalle de confiance, synthèse et recommandations

Synchronisation des données — sources API

Alimentation **automatique** par l'ordonnanceur — planification quotidienne à **02 h 00** (heure de Paris)
Prochaine exécution : 31/07/2026 à 02 h 00 - puis recalcul des prévisions (03 h 00) et génération des alertes

Forcer la synchronisation

ERP Ventes
 GET /api/v2/sales
 Statut : **Opérationnelle**
 4 182 330 lignes intégrées - 1 843 rejets
 Dernière exécution : 30/07 à 02 h 04 (4 min 12 s)

WMS Stocks
 GET /api/v1/stock-levels
 Statut : **Opérationnelle**
 128 470 lignes intégrées - 0 rejet
 Dernière exécution : 30/07 à 02 h 06 (1 min 48 s)

Portail Fournisseurs
 GET /api/v1/purchase-orders
 Statut : **En erreur**
 0 ligne intégrée - délai d'attente dépassé
 Dernière tentative : 30/07 à 02 h 09 (3 essais)

▲ Source « Portail Fournisseurs » injoignable depuis 2 exécutions
Les délais d'approvisionnement affichés datent du 28/07 — fiabilité des prévisions réduite sur les produits concernés.

Historique des exécutions (journalisé — RG-14)

Date d'exécution	Source	Déclencheur	Lignes intégrées	Rejets	Durée	Statut
30/07 02:04	ERP Ventes	Ordonnanceur	4 182 330	1 843	4 min 12 s	Succès
30/07 02:06	WMS Stocks	Ordonnanceur	128 470	0	1 min 48 s	Succès
30/07 02:09	Portail Fournisseurs	Ordonnanceur	0	—	30 s	Échec — API
29/07 14:22	ERP Ventes	Steven L. (forcé)	96 210	12	38 s	Succès
29/07 02:03	ERP Ventes	Ordonnanceur	4 090 112	2 011	4 min 02 s	Succès

Les lignes rejetées (types invalides, références inconnues, doublons — RG-02) sont consultables et exportables source par source.

Steven L. — Administrateur

Maquette 5 Synchronisation : état des sources API, forçage manuel réservé à l'administrateur, historique des exécutions

4.3 Accessibilité (RGAA)

La conception intègre le Référentiel Général d'Amélioration de l'Accessibilité dès la maquette :

- **Contrastes** : palette validée à un ratio $\geq 4,5:1$ pour le texte courant (critère RGAA 3.2) ; le statut des stocks n'est jamais porté par la seule couleur (libellé textuel « OK / Sous seuil / Critique »).
- **Sémantique et alternatives** : balises alt sur toute image porteuse d'information, aria-label sur les boutons iconiques (cloche d'alertes, sélecteur de pays), tableaux de données avec en-têtes th et caption.
- **Navigation clavier** : ordre de tabulation logique, focus visible, liens d'évitement vers le contenu principal ; les graphiques Chart.js sont doublés d'un tableau de données accessible.
- **Formulaires** : étiquettes label explicites, messages d'erreur textuels reliés par aria-describedby, pas de placeholder comme seul libellé.
- **Contrôle** : audit de chaque écran avec l'extension WAVE et le plugin Lighthouse (volet accessibilité) avant chaque fin de sprint ; objectif : zéro erreur bloquante WAVE.

5. Exigences non fonctionnelles

5.1 Sécurité OWASP Top 10

Le tableau ci-dessous liste les bonnes pratiques du moment (OWASP Top 10, édition 2021) et leur déclinaison concrète dans StockPredict :

Risque OWASP	Mesures prévues dans StockPredict
A01 Broken Access Control	Contrôle d'accès par rôle vérifié côté serveur à chaque requête ; refus par défaut ; protection contre les failles IDOR (identifiants de ressources vérifiés par rapport aux droits, jamais de confiance dans l'URL).
A02 Cryptographic Failures	HTTPS/TLS 1.3 partout ; mots de passe hashés bcrypt ; aucun secret en dépôt Git (variables d'environnement).
A03 Injection	Requêtes SQL exclusivement paramétrées (ORM/requêtes préparées) ; validation et échappement des entrées ; protection XSS par échappement systématique côté template.
A04 Insecure Design	Modélisation des menaces dès la conception (le présent dossier) ; principe du moindre privilège ; imports atomiques.
A05 Security Misconfiguration	Images Docker minimales et à jour ; en-têtes de sécurité (CSP, X-Content-Type-Options) ; comptes par défaut désactivés.
A06 Vulnerable Components	Dépendances Python épinglées et scannées (pip-audit) et dépendances front auditées (npm audit) à chaque build.
A07 Identification & Authentication	Verrouillage temporaire du compte après cinq échecs consécutifs (RG-15) ; sessions à expiration ; messages d'erreur génériques.
A08 Software & Data Integrity	Pipeline CI signé, revues de code obligatoires avant fusion.
A09 Logging & Monitoring Failures	Journalisation exhaustive (imports, exports, alertes envoyées, connexions, administration) en base + logs applicatifs horodatés.
A10 SSRF	Aucune récupération d'URL fournie par l'utilisateur ; les appels sortants (API sources, TVA/douane) sont restreints à une liste blanche de domaines.
CSRF (complément)	Jeton JWT transmis par en-tête Authorization (non exposé au CSRF) ; en cas de stockage en cookie, SameSite=Strict et jeton anti-CSRF sur les formulaires à effet de bord.

5.2 Performance

La performance perçue est mesurée avec le **plugin Lighthouse OU WEB PAGE TEST** sur chaque écran, avec les budgets suivants en environnement de recette : score Performance ≥ 90 , First Contentful Paint $< 1,8$ s, Largest Contentful Paint $< 2,5$ s, Total Blocking Time < 200 ms. Côté serveur : réponse API < 300 ms au 95e percentile pour les consultations ; l'ingestion nocturne de plusieurs millions de lignes doit tenir dans la fenêtre 02 h – 03 h grâce au partitionnement et aux insertions par lots (COPY). Les résultats Lighthouse sont archivés à chaque fin de sprint pour suivre les régressions.

Cependant, le projet étant un site interne non dédié à une indexation publique sur les moteurs de recherche, le score Lighthouse ne peut être le principal indicateur de performance.

5.3 Scalabilité (documentée)

Le POC tourne sur un serveur unique (VPS) avec Docker Compose : front Vue.js (build statique), API Python, PostgreSQL, worker de calcul. La trajectoire de montée en charge de 1 à 10 000 utilisateurs est documentée en trois paliers : **palier 1** (POC, ≤ 50 utilisateurs) un serveur, sauvegardes PITR ; **palier 2** (≤ 1000) séparation base/application, réplique PostgreSQL en lecture pour les tableaux de bord, cache des KPI, CDN pour les ressources statiques ; **palier 3** (≥ 10000 , international) orchestration Kubernetes (autoscaling horizontal des services sans état), file de messages pour découpler ingestion et calcul, partitionnement par pays et archivage Iceberg. Kubernetes et le déploiement cloud restent volontairement « sur le papier » au stade POC : l'architecture conteneurisée garantit que cette évolution n'exige pas de réécriture. L'ajout d'une nouvelle source de données reste volontairement simple : l'ingestion suit un motif de connecteurs, chaque source étant décrite par une ligne de configuration (URL, authentification, fréquence, mapping des colonnes). Brancher une API supplémentaire consiste donc à ajouter un connecteur et sa configuration, sans modifier le schéma de données ni le cœur applicatif — la table SYNCHRONISATION traçant indifféremment toutes les sources.

5.4 Maintenabilité

La maintenabilité repose sur une documentation produite au fil de l'eau : le présent dossier de conception, un README d'installation par service, la documentation du pipeline de données (sources, transformations, colonnes ignorées) et un dictionnaire de données généré depuis le schéma PostgreSQL. Le code suit les conventions PEP 8 (linter ruff en CI), est versionné sous Git avec revues croisées entre les cinq développeurs, et les algorithmes de prédiction sont couverts par des tests unitaires ciblés (objectif de couverture $\approx 60\%$, priorité aux fonctions de calcul). Toute décision d'architecture est consignée dans un journal des décisions (ADR) pour faciliter la reprise du projet.

Annexe Livrables joints

Fichier	Contenu
uml_use_case.png	Diagramme UML de cas d'utilisation (Figure 1)
mcd_merise.png	Modèle conceptuel de données Merise (Figure 2)
mld.png	Modèle logique de données relationnel (Figure 3)
mpd_stockpredict.sql	MPD : script SQL PostgreSQL exécutable (schéma complet, contraintes, index, partitions, dimension calendrier et jeu de données minimal)
wf_01_login.png ... wf_05_synchronisation.png	Maquettes fonctionnelles des cinq écrans du POC (dont l'écran de synchronisation)